Transport Layer Formula Sheet (Chapter 3)

1. UDP Checksum Algorithm

The checksum is used to detect bit errors in a received UDP segment. It's an algorithm, not a single formula.

Sender Side:

1. Treat the segment contents (including a pseudo-header with IP addresses) as a sequence of 16-bit integers.
2. Sum all these integers using **1's complement arithmetic**. In this type of addition, any carry-out from the most significant bit must be added back to the result (a "wraparound").
3. The checksum value is the 1's complement (i.e., flip all the bits) of the final sum. This value is placed in the checksum field of the UDP header.

Receiver Side:

1. Compute the 1's complement sum of all received 16-bit integers, *including* the checksum field itself.
2. **Check:** If no bit errors occurred, the result of this sum will be all 1s (i.e., '1111111111111111'). If any bit is a 0, an error has been detected.

2. Reliable Data Transfer (RDT) Performance

Sender Utilization for Stop-and-Wait (rdt3.0)

Calculates the fraction of time the sender is busy transmitting data. This formula shows the inefficiency of stop-and-wait protocols.

$$U_{sender} = \frac{L/R}{RTT + L/R} = \frac{D_{trans}}{RTT + D_{trans}}$$

L: Packet size (in bits).
R: Link bandwidth (in bits per second).
RTT: Round-Trip Time between sender and receiver.
D_{trans}**:** Transmission delay (L/R).
Explanation: This is the ratio of the time the sender spends transmitting the packet (D_{trans}) to the total time of one cycle (transmitting the packet + waiting for the ACK, which takes one RTT).

Sender Utilization for Pipelined Protocols

Calculates the sender utilization when up to N packets can be sent without waiting for an ACK.

$$U_{sender} = \frac{N \cdot (L/R)}{RTT + L/R}$$

N: The window size, or the number of packets in the pipeline.
Explanation: By allowing N packets to be sent per RTT, pipelining can increase the sender's utilization by a factor of N, up to a maximum of 100%.

3. TCP Timeout Calculation (Adaptive Timeout)

TCP dynamically sets its retransmission timeout interval by measuring and smoothing the RTT.

1. Estimated RTT (Exponential Weighted Moving Average)

Calculates a smoothed average RTT that is less susceptible to single, noisy measurements.

$$\text{EstimatedRTT} = (1 - \alpha) \cdot \text{EstimatedRTT} + \alpha \cdot \text{SampleRTT}$$

SampleRTT: The measured time from when a segment is sent until its ACK is received.
α**:** The smoothing factor. A typical value is **0.125**.

2. RTT Deviation

Measures the variability of the RTT to set a proper safety margin.

$$\text{DevRTT} = (1 - \beta) \cdot \text{DevRTT} + \beta \cdot |\text{SampleRTT} - \text{EstimatedRTT}|$$

β**:** The smoothing factor for the deviation. A typical value is **0.25**.

3. Timeout Interval

The final retransmission timeout value.

$$\text{TimeoutInterval} = \text{EstimatedRTT} + 4 \cdot \text{DevRTT}$$

Explanation: The interval is the smoothed average RTT plus a safety margin that grows larger as the RTT becomes more variable.

4. TCP Sender Rate Control

The actual amount of unacknowledged ("in-flight") data a sender can have is the minimum of what is allowed by flow control and congestion control.

Flow Control

Prevents the sender from overwhelming the *receiver's* buffer.

$$\text{LastByteSent} - \text{LastByteAcked} \leq \text{rwnd}$$

rwnd (Receive Window): A value advertised by the receiver in every ACK. It specifies how much free buffer space the receiver has available.

Congestion Control

Prevents the sender from overwhelming the *network*.

$$\text{LastByteSent} - \text{LastByteAcked} \leq \text{cwnd}$$

cwnd (Congestion Window): A sender-side variable that limits the amount of in-flight data. Its value is dynamically adjusted based on perceived network congestion.

Approximate Sending Rate

The relationship between the congestion window and the sender's average throughput.

$$\text{TCP Sending Rate} \approx \frac{\text{cwnd}}{\text{RTT}}$$

How to Use: This shows that to control its sending rate, TCP's primary mechanism is to adjust the size of its 'cwnd'.

cwnd Management Rules (TCP Reno)

Slow Start: At the beginning of a connection, 'cwnd' starts at 1 MSS and grows **exponentially** (doubles every RTT) until it reaches 'ssthresh'.
Congestion Avoidance (Additive Increase): After 'cwnd' reaches 'ssthresh', it grows **linearly** (increases by 1 MSS every RTT).
Response to Timeout (Major Congestion): 'ssthresh' is set to 'cwnd/2'. 'cwnd' is reset to 1 MSS. The connection re-enters Slow Start.
Response to 3 Duplicate ACKs (Minor Congestion): 'ssthresh' is set to 'cwnd/2'. 'cwnd' is also set to 'cwnd/2'. The connection enters Fast Recovery and then Congestion Avoidance (bypassing Slow Start).